Locators

A locator in Playwright is a way to find/identify/locate an element on a web page so we can interact with it and perform action on it

1. Built-in methods (getByMethods)/ User-facing locators
   1)getByRole()
   2)getByText()
   3)getByLabel()
   4)getByPlaceholder()
   5)getByAltText()
   6)getByTitle()
   7)getByTestId()

2. Locator(selector)
   1)css Selector
   2)xpath

1)getByAltText()=It is a Playwright locator method that finds an image (or other element such as vedio,audio,media tag) using its alt attribute means the alternative text that describes the image.
eg:<img src="logo.png" alt="Company logo">

Snytax:page.getByAltText(text, options)
```
page.getByAltText('Company logo')
            or
page.getByAltText('Company logo', { exact: true })
```

{exact:true}= The text must match fully and exactly, including upper/lower case.
{exact: false}=The text can match partially, and upper/lower case is ignored. This is the default.

```
import {test,expect, Locator} from "@playwright/test"
Run | Debug | Inspect
test("getByAltText",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
let logo:Locator=page.getByAltText("Tricentis Demo Web Shop",{exact:true})
await expect(logo).toBeVisible()
})
```

2)getByText() =locate/finds an element by the visible text shown on the page.
eg:<h1>Welcome to our website</h1>
<button>Login</button>

Syntax:page.getByText(text, options)
```
page.getByText('Welcome')
        or
page.getByText('Welcome to our website', { exact: true })
```

```
Run | Debug | Inspect
test.only("getByText",async({page})=>{
await page.goto("https://demowebshop.tricentis.com/")
let text:Locator= page.getByText("Featured products",{exact:true})
await expect(text).toBeVisible()
})
```

3)getByRole()= locates an element using its ARIA role (Accessible Rich Internet Applications),accessible name, and its ARIA attributes.It is the recommended locator because it finds elements the same way a real user sees them.
Arial Role=What the element is (button, link, checkbox, heading, textbox)
Accessible name=The name of the element, usually its visible text or label etc
eg:

```
<button>Login</button>
<a href="/home">Home</a>
<input type="checkbox" id="terms"> <label for="terms">Accept terms</label>
<h1>Welcome</h1>
<input type="text" aria-label="Username">
```

syntax: page.getByRole(role, options)

```
page.getByRole('button', { name: 'Login' })
        or
page.getByRole('checkbox', { name: 'Accept terms'})
```

```
test.only("getByRole",async({page})=>{
    await page.goto("https://demowebshop.tricentis.com/")
    await page.getByRole('textbox', { name: 'q' }).click();
    await page.getByRole("link",{name:'Register'}).click();
    await page.getByRole("radio", { name: 'Male', exact: true }).click();
     await page.getByRole("radio", { name: 'Female', exact: true }).click();
     await page.getByRole("textbox", { name: 'First name'}).fill("12345");
     await page.waitForTimeout(3000)
```

4)getByLabel()=locates an input element using its associated <label> text.
eg:

```
<label for="username">Username</label>
<input type="text" id="username">
```

Notes:The for value in the label and the id value in the input must be the same. That is how getByLabel() knows which input belongs to which label.

syntax:page.getByLabel(text, options)

```
page.getByLabel('Username')
        or
page.getByLabel('Username', { exact: true })
```

```
test("getByLabel",async({page})=>{
    await page.goto("https://demowebshop.tricentis.com/")
    await page.getByRole("link",{name:'Register'}).click();
    await page.getByLabel("First name:").fill("Amruta")
    await page.getByLabel("Last name:").fill("R")
    await page.getByLabel("Email:").fill("abc@gmail.com")
})
```

5)getByPlaceholder()=finds/locate an input box using its placeholder text.
eg:
<input type="text" placeholder="Enter username">
syntax:page.getByPlaceholder(text, options)
page.getByPlaceholder('Enter username')
or
page.getByPlaceholder('Enter username', { exact: true })

```
test("getByplaceHolder",async({page})=>{
    await page.goto("https://www.flipkart.com/")
    await page.getByPlaceholder("Search for Products, Brands and More").first().fill("iphone")
    await page.waitForTimeout(2000)
})
```

6)getByTitle()=finds/locate an element using its title attribute
eg:
?
syntax:page.getByTitle(text, options)
page.getByTitle('Help page')
or
page.getByTitle('Help page', { exact: true })

```
test("getByTitle",async({page})=>{
    await page.goto("https://demowebshop.tricentis.com/")
    await page.getByTitle("Show details for Build your own cheap computer").first().click()
    await page.waitForTimeout(2000)
})
```

7)getByTestId()=finds an element using its test ID attribute (by default it will check for data-testid).
Customizable: By default it looks for data-testid, but you can change it to any attribute ,such as data-qa or data-test, by setting testIdAttribute in playwright.config.ts.
eg:
<button data-testid="submit-btn">Submit</button>
syntax:page.getByTestId(testId)
page.getByTestId('submit-btn')

One more case: if your app uses a different attribute, such as data-test or data-qa, getByTestId() will not find the element until you customize it:

```
// playwright.config.ts
   use: {
   testIdAttribute: 'data-test'
   }
```

```
test("getByTestId",async({page})=>{
    await page.goto("https://www.saucedemo.com/")
    await page.getByTestId("username").fill("amruta")
    await page.waitForTimeout(2000)
})
```